

CS683 Project Assignment

iperf3 network tester
Jerold (Jerry) Abramson

1. Overview	3
1.1. More Details	3
2. Video Presentation	3
3. Functional Requirements and limitations	3
3.1. Challenges resolved	4
3.2. Future Goals	4
4. Motivation	4
5. Purpose	4
5.1. Potential Users	4
5.2. Related Work	5
5.2.1. Text-Based Mode	5
5.2.2. “Fancy” line drawing and fonts	5
5.2.3. Similar Android Applications	6
5.2.4. Similar iOS application	6
6. Rall Requirements have been completed	6
6.1. Requirement 1	6
6.2. Requirement 2 – User Interface implemented	7
6.3. Requirement 3	7
6.4. Requirement 4 – Requirement removed due to further application analysis.	7
7. Design and Implementation	8
7.1. Layered Architecture	8
7.2. Test Execution Flow	8
7.3. Full component breakdown organized by prose	8
2. Project Structure	9
8.	9
9. Screen shots	10

9.1.	Real Android Phone	10
9.2.	Android Emulator	13
	Timeline	26
10.		26
11.	Current Status - Complete	27
11.1.	Fallback simulation	27
12.	AI Usage Log	28
13.	Future Work (Optional)	28
•	Project Demo Links	28

1. Overview

The application being developed will utilize the de-facto network protocol tool, **iperf3**, for testing network speeds between a client and a server.

1.1. More Details

- For my project I am implementing a network performance monitor based on the iPerf3 protocol.
- Although there are a couple of Android apps that already partially implement this, one of them is terribly complex, quite expensive, (**Analti**), more focused on Wi-Fi and internet performance monitoring.
- There is also a very old version that is nothing more than the text-based version of the existing 'iperf3' application that runs in a console terminal window.
- There is a nice version that runs on iOS, but that has not been ported to Android. However, the iOS version sacrifices details for ease of use and a clean user interface.
- Here are the associated application icons:

My Project	Analti by analiti Networking Experts	Iperf3 by Uncle Tools
TBD	https://play.google.com/store/apps/details?id=com.analiti.fastest.android	https://play.google.com/store/apps/details?id=tools.uncle.network
		

2. Video Presentation

I broke up the video presentation into 4 videos, roughly covering different aspects of the deliverables.

The videos are unlisted public links:

Video Description	YouTube link
CS683 - Project - Part 1 of 4 - Jerold Abramson	https://youtu.be/duv60JoqVTs
CS683 - Project - Part 2 of 4 - Jerold Abramson	https://youtu.be/5vFkzoGIDoU
CS683 - Project - Part 3 of 4 - Jerold Abramson	https://youtu.be/2TtQ8dWUnhg
CS683 - Project - Part 4 of 4 - Jerold Abramson	https://youtu.be/iXRrtdUoofw

3. Functional Requirements and limitations

- The application **will only be implementing** the client-side functionality of iPerf3:
 - It will assumed that an **iPerf3** server is running on another host.
 - I will be providing a mechanism to ensure that a server will be available to the instruction team during analysis and grading.
- The actual **iperf3** protocol **will not be** reimplemented.

3.1.Challenges resolved

- I have solved working through the mechanism to ensure that the `iperf3` binary is uploaded to the Android device/emulator.
- I have also solved having the `iperf3` binary being executable inside the standard Android sandbox security architecture.
- I have identified the core issue with executing native executables on Android which is specific to SELinux.
- I have identified a reasonable resolution for this for the project assignment.
- I am using the standard Kotlin/Java ProcessBuilder class APIs.
- I have a script that can be run by the testing team to allow for the `iperf3` executable to run.
- The basics of the workaround are to disable SELinux on the emulator, which allows for the binaries included with the application to be executable.

3.2.Future Goals

- I have studied a GitHub project that incorporates the `iperf3` binary into a full Android NDK application.
- I even moved on further and started converting my application to utilize this approach.
- The native `iperf3` native libraries (`.so`) **will not be** utilized using direct JNI APIs from Kotlin/Android.
- The official releases of the `iperf3` binary for Android (all hardware architectures and emulators) will be utilized for this project.
- I have put that effort into another branch:
 - <https://github.com/CS683/project-jerryabramson/tree/tryembeddediperf3>
- For the course project I will continue with the current approach of using an external `iperf3` binary.

4. Motivation

I utilize network testing tools as part of my home-labbing hobby.

One of the tools I am constantly utilizing is based on the `iPerf3` protocol, which is defined here:

The full `iperf3` source code and technical documentation for `iperf3` is available on this website:

- <https://iperf.fr/>

The full source code and technical documentation for the `iperf3` implementation on Android is available here:

- <https://github.com/davidBar-On/android-iperf3/>

I utilize network testing tools as part of my "home-labbing" hobby.

- One of the tools I am constantly utilizing is based on the `iPerf3` protocol, which is defined here:
- *For some context on `iPerf3` is available → here*

5. Purpose

Provide the ability to test WiFi and generalized network performance using an Android phone.

5.1.Potential Users

- `iperf3` is a well-known tool for networking professionals.
- My goal is to launch this application on the Android Play Store over the summer for a one-time purchase price of `$0.99`

5.2.Related Work

I have developed a **Java text-based console program** that runs the **iperf3** native executable in the background and provides performance details in an asynchronous manner.

The application supports running at the command-line with ‘fancy’ ANSI line drawing and fonts, as well as standard text-based output suitable for scripts.

Here are some examples of this application running in standard text-based mode

- Note: **pvc** is a local hostname reachable in my LAN.

5.2.1. Text-Based Mode

```
TERM=dumb LANG=C newTestBandWidth.sh pvc -R
[Sat Mar 28 13:10:52 EDT 2026] Executing Command-line: '/opt/local/bin/iperf3' '--forceflush' '--connect-timeout' '3000' '-c' 'pvc' '-O 2' '-P 8' '-t' '10' '-R'
Initiating [ .....Connecting to host pvc, port 5201
Reverse mode, remote host pvc is sending
  Local Host/IP: 192.168.1.10 remote Host/IP: 192.168.1.7 Remote Port: 5201
...XXXXX Skipping 0.00-1.01          2.35 Gbits/sec (omitted)
...XXXXX Skipping 1.01-2.00          2.35 Gbits/sec (omitted)
...XXXXX Running 0.00-1.01           2.35 Gbits/sec
...XXXXX Running 1.01-2.01           2.35 Gbits/sec
...XXXXX Running 2.01-3.00           2.35 Gbits/sec
...XXXXX Running 3.00-4.01           2.35 Gbits/sec
...XXXXX Running 4.01-5.01           2.35 Gbits/sec
...XXXXX Running 5.01-6.00           2.35 Gbits/sec
...XXXXX Running 6.00-7.01           2.35 Gbits/sec
...XXXXX Running 7.01-8.00           2.35 Gbits/sec
...XXXXX Running 8.00-9.00           2.35 Gbits/sec
Running 9.00-10.01          2.35 Gbits/sec
Results: 0.00-10.01          2.35 Gbits/sec sender
=> 0.00-10.01          2.35 Gbits/sec receiver

[Avg]      2.35 Gbits/sec
[Min]      2.35 Gbits/sec
[Max]      2.35 Gbits/sec

-----
Return Code: 000 [Avg=2.35 Gbits/sec]
```

5.2.2. “Fancy” line drawing and fonts

```
newTestBandWidth fedoranas -R
[Sat Mar 28 13:16:38 EDT 2026] Executing Command-line: 'C:/Program Files/iperf3.12_64/iperf3.exe' '--forceflush' '--connect-timeout' '3000' '-c' 'fedoranas' '-O 2' '-P 8' '-t' '10' '-R'
Connecting to host fedoranas, port 5201
Reverse mode, remote host fedoranas is sending
  Local Host/IP: 192.168.1.20 remote Host/IP: 192.168.1.204 Remote Port: 5201
Running 1.00-2.00 - ( [ 7.61 Gbits/sec

newTestBandWidth fedoranas -R
[Sat Mar 28 13:16:38 EDT 2026] Executing Command-line: 'C:/Program Files/iperf3.12_64/iperf3.exe' '--forceflush' '--connect-timeout' '3000' '-c' 'fedoranas' '-O 2' '-P 8' '-t' '10' '-R'
Connecting to host fedoranas, port 5201
Reverse mode, remote host fedoranas is sending
  Local Host/IP: 192.168.1.20 remote Host/IP: 192.168.1.204 Remote Port: 5201
Results: 0.00-10.00 [ 7.31 Gbits/sec sender
=> 0.00-10.00 [ 7.31 Gbits/sec receiver

[Avg] [ 7.31 Gbits/sec
[Min] [ 5.59 Gbits/sec
[Max] [ 9.23 Gbits/sec

Return Code: 000 [Avg=7.31 Gbits/sec]
```

5.2.3. Similar Android Applications

There are a few similar implementations of this tool on the Android play store, shown in the below table:

App Name	Play Store Link
Analti <i>analiti Networking Experts</i>	https://play.google.com/store/apps/details?id=com.analiti.fastest.android
iperf3 <i>Uncle Tools</i>	https://play.google.com/store/search?q=iperf3&c=apps

5.2.4. Similar iOS application

App Name	App Store Link
iPerf3 WiFi Speed Test	https://apps.apple.com/us/app/ipperf-3-wifi-speed-test/id1462260546

6. Rall Requirements have been completed

The status'es have all been updated to reflect which iteration the requirement was completed in.

6.1.Requirement 1

[illegible]

6.2.Requirement 2 – User Interface implemented

Title	User Interface Displays Interaction and Status
Description	After the user presses the button ‘Run Iperf-3 Test’ the execution runs in the background, with the user interface providing a progress bar.
Mockups	
Acceptance Tests	The progress bar moves forward while the background job is running.
Test Results	The progress bar moves forward, and the ‘Run Iperf-3 Test’ button is greyed out. Once the background job is completed, the progress bar shows ‘Complete’ and the button is again usable (no longer greyed out)
Status	Iteration 1 – Job runs in background Iteration 2 – As the job is running in the background, the progress bar is updated with the most recent bandwidth result i.e [=====] Interval 5: 393 Mbits/sec Iteration 2 – Accurate results based on the optional inputs (i.e. hostname, run count, parallel streams, etc...) Iteration 3 – All typical iperf3 options provided (duration, parallel streams, upload/download, omitted tests)

6.3.Requirement 3

Title	Server Address and other options
Description	The user shall be able to type in a valid server address of a remote iperf3 server
Mockups	
Acceptance Tests	host-name/IP Address validation
Test Results	Positive Test: "localhost" Test runs Negative Test: "abcd.baddomain" <i>Validation error</i>
Status	Iteration 1 - Allow entry of server name Iteration 2 - Validates remote host before execution Iteration 3 - perform iperf3 test with hard-coded-count of 10 against provided host Iteration 4 – All command-line arguments supported: • Server Address • Server Port • Transmit Mode (Upload/Download) • Parallel Streams • Skip Iteration Count • Test Duration • Output Units – No longer makes sense for a mobile solution. • Loop count – Will be implemented for play store release. • Share as...button saves output using a persistent data store based on room database – Will be implemented for subsequent play store release.

6.4.Requirement 4 – ~~Requirement removed due to further application analysis.~~

Title	Unit of measurement
Description	The user shall be able to specify the unit of measurement for output bandwidths
Mockups	Entry of MB, Entry of gb, Entry of bytes – Requirement no longer applicable.
Acceptance Tests	Validate rationale unit text
Test Results	Positive: MB, Negative foobar

7. Design and Implementation

I utilized the new <https://claude.ai/> capability for render of diagrams without the need for Canvas, PlantUML, or other tools.

AI Prompt:

Design the basic architecture for an Android application that performs iperf3 tests against a remote iperf3 server

7.1. Layered Architecture

- The app is organized into four tiers from UI down to the native iperf3 binary.
- Unlike the AI provided Architecture, **I will not be utilizing an NDK/JNI Bridge** but instead will be executing the iperf3 binary and parsing its output.
- I decided to focus more effort on the video presentation.
- I will also be rearchitecting a solution to utilize JNI bindings and the Android Native Development Kit (NDK)

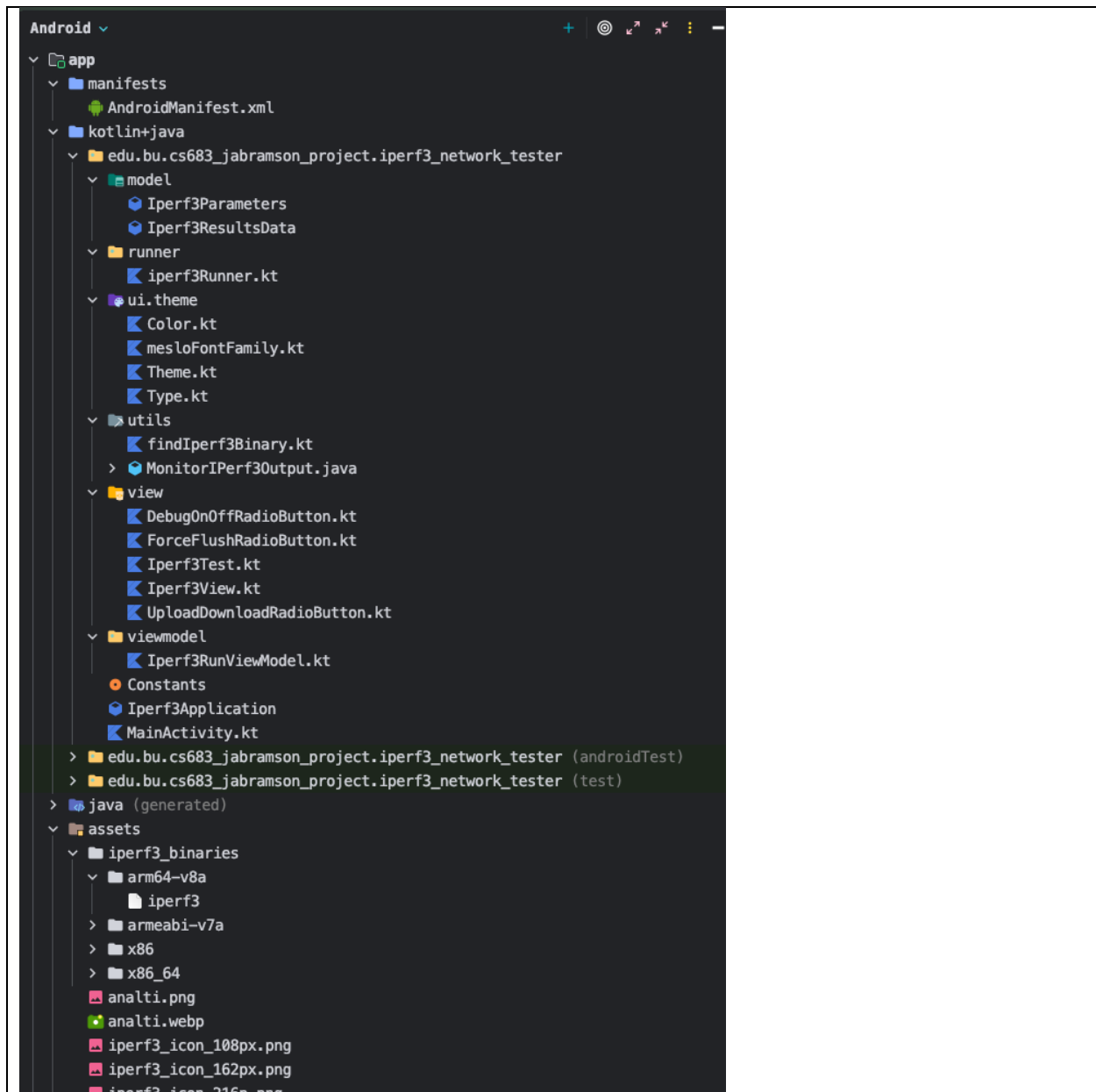
7.2. Test Execution Flow

Instead of utilizing the JNI bridge, I will be using the standard Kotlin/Java ProcessBuilder APIs to run the iperf3 binary in the background.

7.3. Full component breakdown organized by prose

I will be rewriting this section sometime after course completion.

2. Project Structure



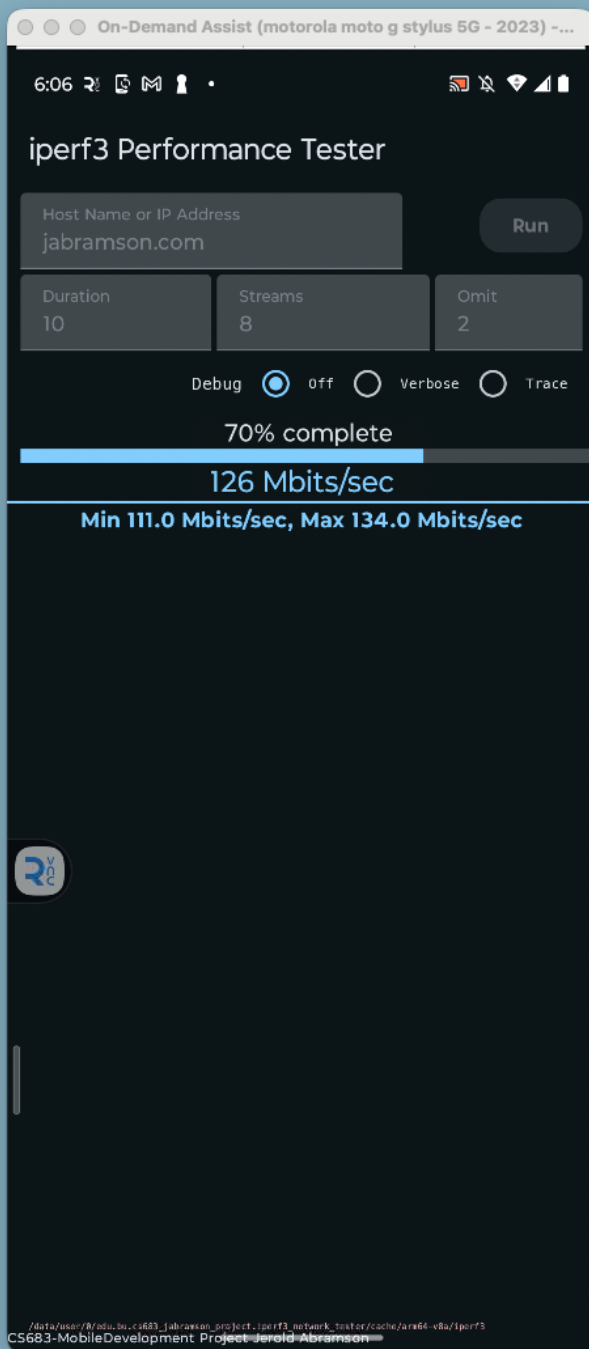
8.

9. Screen shots

9.1.Real Android Phone







9.2.Android Emulator

5:59



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

10

Streams

8

Omit

2

Debug



Off



Verbose



Trace

Launching iperf3 ...

iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

10

Streams

8

Omit

2

Debug



Off



Verbose



Trace

19% complete

66.4 Mbits/sec

Min , Max

5:59   



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

10

Streams

8

Omit

2

Debug



Off



Verbose



Trace

60% complete

186 Mbits/sec

Min 161.0 Mbits/sec, Max 186.0 Mbits/sec

5:59   



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

10

Streams

8

Omit

2

Debug



Off



Verbose



Trace

90% complete

176 Mbits/sec

Min 161.0 Mbits/sec, Max 186.0 Mbits/sec

5:59



iperf3 Performance Tester

Host Name or IP Address

Run

Duration

Streams

Omit

Direction ☒ Download ☐ Upload

Debug ☒ Off ☐ Verbose ☐ Trace

Average: 179.0 Mbits/sec

Maximum: 188.0 Mbits/sec

Minimum: 161.0 Mbits/sec

5:58   



iperf3 Performance Tester

Host Name or IP Address
badhost.baddomain

Run

Duration
30

Streams
1

Omit
0

Direction ☒ Download ☐ Upload

Debug ☐ Off ☐ Verbose ☒ Trace

Return Code: 1

```
iperf3: error - unable to connect to server: No address  
associated with hostname
```

5:57   



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

30

Streams

1

Omit

0

Debug



Off



Verbose



Trace

6% complete

148 Mbits/sec

Min 148.0 Mbits/sec, Max 154.0 Mbits/sec

5:58   



iperf3 Performance Tester

Host Name or IP Address

1.1.1.1

Run

Duration

30

Streams

1

Omit

0

Direction ☒ Download ☐ Upload

Debug ☐ Off ☐ Verbose ☒ Trace

Return Code: 1

iperf3: error - unable to connect to server: Connection timed out

5:57



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

30

Streams

1

Omit

0

Debug



Off



Verbose



Trace

53% complete

165 Mbits/sec

Min 98.7 Mbits/sec, Max 165.0 Mbits/sec

Connecting to host jabramson.com, port 5201
Reverse mode, remote host jabramson.com is sending
Local Host/IP: 10.0.2.16
Remote Host/IP: 192.168.1.32
Remote Port: 5201

Running	0.00-1.00	154 Mbits/sec
Running	1.00-2.00	148 Mbits/sec
Running	2.00-3.00	129 Mbits/sec
Running	3.00-4.00	119 Mbits/sec
Running	4.00-5.00	100 Mbits/sec
Running	5.00-6.00	107 Mbits/sec
Running	6.00-7.00	111 Mbits/sec
Running	7.00-8.00	98.7 Mbits/sec
Running	8.00-9.00	115 Mbits/sec
Running	9.00-10.00	133 Mbits/sec
Running	10.00-11.00	138 Mbits/sec
Running	11.00-12.00	134 Mbits/sec
Running	12.00-13.00	135 Mbits/sec
Running	13.00-14.00	135 Mbits/sec
Running	14.00-15.00	160 Mbits/sec
Running	15.00-16.00	165 Mbits/sec

5:57   



iperf3 Performance Tester

Host Name or IP Address

jabramson.com

Run

Duration

30

Streams

1

Omit

0

Debug ☐ Off ☒ Verbose ☐ Trace

30% complete

115 Mbits/sec

Min 98.7 Mbits/sec, Max 154.0 Mbits/sec

```
Connecting to host jabramson.com, port 5201
Reverse mode, remote host jabramson.com is sending
  Local Host/IP: 10.0.2.16
  Remote Host/IP: 192.168.1.32
  Remote Port: 5201
```

5:56   



iperf3 Performance Tester

Host Name or IP Address

Run

Duration

Streams

Omit

Direction ☐ Download ☒ Upload

Debug ☒ Off ☐ Verbose ☐ Trace

Average: 144.0 Mbits/sec

Maximum: 151.0 Mbits/sec

Minimum: 134.0 Mbits/sec

5:54   



iperf3 Performance Tester

Host Name or IP Address
jabramson.com

Run

Duration
10

Streams
8

Omit
2

Debug ☒ Off ☐ Verbose ☐ Trace

30% complete

149 Mbits/sec

Min 149.0 Mbits/sec, Max 149.0 Mbits/sec

10. Timeline

Iteration	Application Requirements • Essential • Desirable • Optional	Android Components and Features to be used
0.5	Redone to utilize Jetpack Compose	Jetpack Compose
1 DONE and thrown away	Essential: Stubbed version of progress bar	TBD (Networking)
COMPLETED 1	Essential: Application executes in background against a real host using the iperf3 executable	TextView
2 COMPLETED	As the iperf3 test is running in the background, provide a running total of the current bitrate	
2	After the test completes, provide a Min, Max, and Avg bitrate	
3 COMPLETED Test harness is in development	Ensure all possible network issues are handled gracefully	
2	Essential: Manual keyboard entry of test count	TextView
1		
2 or 3 COMPLETED EARLY Iteration 1	Essential: perform a single iperf3 test (count=1)	TBD (Consuming iperf3 native executable)
2 Handling iperf3 execution errors is adequate. Completed in iteration 4.	Desirable: Validate remote host	Networking
3 – DONE EARLY	Essential: Perform iperf3 to specified host with a count of 1	TBD
4 – DONE EARLY	Essential: iperf3 runs against specified host for specified count with specified unit	TBD

11. Current Status - Complete

- For the best experience use one of the simulators below.
- I have built in a simulation in case you have issues.
- I have also put together a small script that is executed prior to build.
 - This script changes the Android permissions to disable SELinux.
 - This will only work for certain simulators (include those shown below).
 - It will also work for root'ed Android devices.

If it is necessary to run the commands manually, these are the required steps:

```
$ adb -s ZD222CYWBG shell
genevn:/ $ su
genevn:/ # getenforce
Enforcing
genevn:/ # setenforce 0
genevn:/ # getenforce
Permissive
genevn:/ #
```

macOS: :Pixel 6 Android 16.0 ("Baklava") | arm64

Windows: Pixel 6 Android 16.0 ("Baklava") | x86_64



11.1. Fallback simulation

- If the test cannot execute an appropriate iperf3 executable, a simulation using actual iperf3 output is utilized instead.
- Additionall, if the hostname 'testing' is entered, the simulation is run.

12. AI Usage Log

All AI usage was tracked inside of the both the code and documentation directories as markdown files.

```
./Code/Iperf3NetworkTester/planning.md
./Code/Iperf3NetworkTester/app/src/main/structure.md
./Code/Iperf3NetworkTester/commandLineBuildApp.md
./Code/Iperf3NetworkTester/AGENTS.md
./Code/Iperf3NetworkTester/session-ses_2ba1.md
./Code/Iperf3NetworkTester/IPERF3_EXE.md
./Doc/02_answer.md
./Doc/iperf3Bundling/gradleTips.md
./Doc/iperf3Bundling/AGENTS.md
./Doc/android_icon_steps.md
./Doc/03_question.md
./Doc/01_question.md
./Doc/pre-deploy.md
./AGENTS.md
./CLAUDE.md
```

13. Future Work (Optional)

1. Rework application to utilize NDK and JNI bindings. I have already put together some prototypes of this solution.
2. Deploy to Android play store and begin marketing/sales.
3. Enhance video presentations and attempt to start a YouTube channel.

• Project Demo Links

Video Description	YouTube link
CS683 - Project - Part 1 of 4 - Jerold Abramson	https://youtu.be/duv60JoqVTs
CS683 - Project - Part 2 of 4 - Jerold Abramson	https://youtu.be/5vFkzoGIDoU
CS683 - Project - Part 3 of 4 - Jerold Abramson	https://youtu.be/2TtQ8dWUnhg
CS683 - Project - Part 4 of 4 - Jerold Abramson	https://youtu.be/iXRrtdUoofw

• References